

Optimization for LLM Inference Configuration: A Hardware-Aware Multi-Objective Tuning Framework

Ronak Vimal

June 21, 2026

Abstract

Picking a good runtime configuration for an LLM serving stack is harder than it looks. Throughput, time-to-first-token (TTFT) and time-per-output-token (TPOT) all matter, and they trade off against each other in a way that depends on coupled choices: tensor-parallel width, batching limits, KV-cache layout, scheduling, GPU memory placement, and CPU/disk offloading. We treat this as a hardware-aware multi-objective optimization problem and focus on two largely independent decision spaces: a *vLLM Serving-Runtime* space (Ω_S , nine mixed-type knobs) and a *FlexGen Memory-Topology* space (Ω_T , eight enumerated knobs plus nine continuous placement fractions).

The main contribution is **SCOOT-qNEHVI**, a small but consequential modification of SCOOT [4] in which the original HEBO + EHVI optimizer is swapped out for a BoTorch mixed-space Gaussian process with the *q-Noisy Expected Hypervolume Improvement* acquisition [2, 7]. We then run a head-to-head comparison of five methods on Ω_S — vLLM defaults, random search, an LLM-driven agent, the original SCOOT, and our SCOOT-qNEHVI — across four GPU allocations on Llama-2-7B-Chat. SCOOT-qNEHVI wins or ties most per-objective contests and improves on the vLLM default by up to +10% throughput, −28% TTFT, and −31% TPOT. On Ω_T we re-confirm a result from FlexGen’s authors: the two-level (outer enumeration + inner LP) search beats a 3,375-point grid by 14–31% in predicted per-token latency, on four memory-pressured language models. Finally, we explore the *Cartesian product* of both methods’ Pareto fronts on Llama-2-13B-Chat; the joint pick beats the vLLM default on every measured objective on both allocations we tested. Code, scripts, raw results, and figure-generation utilities are at

github.com/Kytamnh/optimization-llm-inference.

1 Introduction

Once a large language model is trained, the cost story shifts almost entirely to inference: every served token costs GPU time, every queue backlog costs latency. Modern engines like vLLM [12] or FlexGen [16] expose dozens of runtime knobs that change those costs in non-trivial ways. The list of things you can tune is long: the tensor-parallel (TP) world size, the maximum number of concurrent sequences, the per-iteration token budget, the KV-cache block size, a scheduler delay factor, prefix-cache reuse, the placement of weights / KV cache / activations across the GPU/CPU/disk hierarchy, the compression mode, and so on. The frustrating part is that these knobs do not move independently. Pushing throughput tends to hurt first-token latency. Saving GPU VRAM tends to add PCIe traffic. Spreading the model across more GPUs relieves memory pressure but pays for it in all-reduce communication.

Because of these couplings, a configuration that is great on a 24-GiB RTX A5000 is rarely a good fit for a 48-GiB A6000, let alone a 141-GiB H200. Hand-tuning a serving stack on every (model, hardware, workload) tuple is therefore not a serious plan. The community has answered with a small zoo of automatic-tuning approaches: vLLM ships defaults that are intentionally workload-agnostic; SCOOT [4] uses multi-objective Bayesian optimization; FlexGen [16] attacks the offloading sub-problem with an analytical cost model and a linear program.

Contributions.

1. **SCOOT-qNEHVI.** We modify SCOOT by replacing its HEBO + EHVI optimizer with a BoTorch mixed-space Gaussian process and the noise-aware *q-Noisy Expected Hypervolume Improvement* acquisition [7].
2. **Five-method study on Ω_S .** We compare vLLM defaults, random search, an LLM agent, the original SCOOT, and SCOOT-qNEHVI on Llama-2-7B-Chat across four 2–4-GPU allocations and the ShareGPT workload.
3. **LP vs. grid on Ω_T .** We compare FlexGen’s two-level search to a uniform 3,375-point grid baseline across four memory-pressured language models, confirming the LP’s advantage.
4. **Cartesian-product Pareto.** We propose and evaluate a *Cartesian-product* method that combines the two methods’ Pareto fronts on Llama-2-13B-Chat, discuss what it does and does not buy us, and outline a properly nested optimizer as future work.

The rest of the paper is laid out as follows. Section 2 reviews the related literature. Section 3 states the optimization problem mathematically. Section 4 describes the five Ω_S methods, the two Ω_T methods, and the joint Cartesian step. Section 5 covers the experimental environment and Section 6 presents the empirical results. Section 7 discusses what worked and what did not, and Section 8 concludes with a sketch of the nested extension.

2 Background and Related Work

2.1 LLM Inference Engines and Memory Hierarchy

A modern LLM serving engine sits on top of a layered memory system: GPU VRAM, host RAM, and (sometimes) disk. vLLM [12] introduced *PagedAttention* for KV-cache management, allocating KV blocks in fixed-size pages (the `block-size` knob), which avoids the classic internal fragmentation of pre-allocated KV buffers. The vLLM scheduler can be tuned for different traffic patterns: there are limits on concurrent sequences (`max_num_seqs`) and on per-iteration tokens (`max_num_batched_tokens`), an option for chunked prefill, prefix caching, and a delay knob that lets the scheduler grow batches more aggressively.

FlexGen [16] takes the opposite extreme: when the model is too large to live on the GPU, it offloads weights, KV cache, and activations across the GPU/CPU/disk tiers. The system’s analytical cost model predicts per-block latency from two sources of decisions: the outer-policy variables (GPU batch size, number of GPU batches, compression mode, CPU compute delegation, IO/compute overlap) and the continuous placement fractions $(w_g, w_c, w_d), (c_g, c_c, c_d), (h_g, h_c, h_d)$ for weights, KV cache and activations.

A short list of related serving systems includes DeepSpeed-Inference [1], FlashAttention [6], and AlpaServe [13]. We focus on vLLM and FlexGen because they expose the cleanest, most orthogonal parameter spaces of the three for optimization purposes.

2.2 Bayesian Optimization for Hyperparameters

Bayesian Optimization [10, 15, 17] fits a probabilistic surrogate (most often a Gaussian process [14]) to the function being optimized and chooses the next query point by maximizing an *acquisition* function that trades exploration for exploitation. Random search [3] remains a tough baseline because it makes no assumptions about the landscape. For multi-objective problems, the canonical acquisition is *Expected Hypervolume Improvement* (EHVI) [9, 11], which scores a candidate by the expected gain in dominated hypervolume. The noise-aware variant *qNEHVI* [7] extends this to noisy and batched settings; the more recent *qLogNEHVI* [8] fixes some numerical pathologies. **HEBO** [5] is the BO backend that ships with the original SCOOT; it adds heteroscedastic noise modelling and input warping. **BoTorch** [2] is the framework we use for our qNEHVI implementation.

2.3 LLM-as-Optimizer

There is also a recent line of work that uses an LLM itself as a configuration generator: feed the model the metric history so far, and ask for the next configuration in natural language [19]. We include exactly such an agentic baseline in Section 4 for fair empirical comparison.

2.4 LLM Serving Tuning

The closest prior work is **SCOOT** [4] (WWW 2025). It frames LLM serving as a constrained multi-objective optimization, warm-starts with Sobol QMC samples, fits a multi-output GP, learns a random-forest *hidden-feasibility* classifier, and picks the next configuration with EHVI. The SCOOT optimizer is built on HEBO. Our SCOOT-qNEHVI swaps that out for a BoTorch mixed-space GP plus qNEHVI; that swap is the central contribution of this work.

3 Problem Formulation

3.1 Configuration Spaces

We use Ω_S for the *vLLM Serving-Runtime* configuration space and Ω_T for the *FlexGen Memory-Topology* configuration space. Table 1 lists the nine knobs in Ω_S , taken from the SCOOT formulation [4]. Table 2 does the same for Ω_T .

3.2 Objectives

For a configuration x and a stress-test of M requests over duration D , write $t_{j,\text{arr}}, t_{j,\text{first}}, t_{j,\text{last}}$ for the arrival, first-token and last-token timestamps of request j and m_j for its number of output tokens. The three objectives are:

$$T(x) = M/D \quad (\text{req/s, maximize}), \quad (1)$$

$$\Phi(x) = \frac{1}{M} \sum_{j=1}^M (t_{j,\text{first}} - t_{j,\text{arr}}) \quad (\text{ms, minimize}), \quad (2)$$

$$\Theta(x) = \frac{1}{M} \sum_{j=1}^M \frac{t_{j,\text{last}} - t_{j,\text{first}}}{m_j - 1} \quad (\text{ms, minimize}). \quad (3)$$

Table 1: Ω_S , the vLLM Serving-Runtime configuration space. Nine mixed-type knobs; once continuous ranges are discretized, $|\Omega_S| \approx 9.6 \times 10^7$.

Parameter	Search space	What it controls
tensor-parallel	$\{1, 2, 4\}$	GPUs combined for one model instance; trades model memory for all-reduce traffic.
max-num-seqs	powers of 2, 64–8192	Maximum concurrent sequences; controls request-level concurrency and KV pressure.
max-num-batched-tokens	token budget, $\leq \max(8192, 2L_{\max})$	Tokens batched per iteration; affects GPU occupancy.
block-size	$\{8, 16, 32\}$	PagedAttention KV block granularity.
enable-chunked-prefill	boolean	Splits long prompts into chunks; can prevent decode starvation.
scheduler-delay-factor	$\{0.0, 0.2, \dots, 2.0\}$	Wait time to grow batches; trades TTFT for throughput.
enable-prefix-caching	boolean	Reuses KV cache across shared prefixes.
disable-custom-all-reduce	boolean	Switches NCCL all-reduce implementation.
use-v2-block-manager	boolean	Selects vLLM’s block manager.

Table 2: Ω_T , the FlexGen Memory-Topology configuration space. Five enumerated decisions plus three continuous placement vectors that each sum to one ($\dim(\Omega_T) = 5 + 9$).

Parameter	Type	System meaning
GPU batch size b	integer	Samples per GPU batch.
Number of GPU batches n	integer	GPU batches grouped per block; weights are reused.
Effective block size bn	derived	Controls amortization of weight I/O.
(w_g, w_c, w_d)	continuous, $\sum = 1$	Weights placement fractions on GPU/CPU/disk.
(h_g, h_c, h_d)	continuous, $\sum = 1$	Activation placement fractions.
(c_g, c_c, c_d)	continuous, $\sum = 1$	KV-cache placement fractions.
CPU compute delegate	boolean	Run attention scores on CPU when KV is there.
Compression mode	enum $\{\text{fp16}, \text{int4}\}$	Reduces footprint and I/O.
I/O–compute overlap	boolean	Hides I/O behind compute.

3.3 Multi-Objective Constrained Optimization

The serving-side problem on Ω_S takes the standard form

$$\min_{x \in \Omega_S} (-T(x), \Phi(x), \Theta(x)) \quad \text{s.t. } c_i(x) \leq 0 \ \forall i, \quad POF(x) \geq \Delta, \quad (4)$$

where the c_i are known constraints (for instance, $\text{max-num-batched-tokens} \geq \text{max-num-seqs}$) and POF is the *probability of feasibility*, estimated from observed runtime failures. A configuration x *dominates* x' if $f_k(x) \leq f_k(x')$ for all k with at least one strict inequality. The set of non-dominated configurations is the *Pareto frontier* $\mathcal{P}$. Given a reference point r , the *hypervolume* of $\mathcal{P}$ is

$$HV(\mathcal{P}, r) = \text{Vol}\left(\bigcup_{x \in \mathcal{P}} \{y \in \mathbb{R}^k : f(x) \preceq y \preceq r\}\right), \quad (5)$$

i.e. the dominated k -dimensional volume below r [20]. We keep the three objectives separate rather than collapsing them into a weighted sum: TTFT and TPOT have very different scales and they

often pull in opposite directions under load [4].

3.4 Topology Optimization Sub-Problem

For a fixed FlexGen outer enumeration $e \in \{b\} \times \{n\} \times \{q\} \times \{\text{delegate}\} \times \{\text{overlap}\}$, the inner placement problem is the linear program

$$\min_{w,h,c} \quad t_{\text{block}}(e, w, h, c) \quad (6)$$

$$\text{s.t.} \quad w \cdot W + c \cdot KV(b, n) + h \cdot A \leq \rho \cdot V_{\text{GPU}}, \quad (7)$$

$$w_g + w_c + w_d = 1, \quad c_g + c_c + c_d = 1, \quad h_g + h_c + h_d = 1, \quad (8)$$

$$0 \leq w_*, c_*, h_* \leq 1, \quad (9)$$

where W , A , and $KV(b, n)$ are the per-layer weight, activation and KV-cache sizes; V_{GPU} is GPU VRAM; and $\rho \in (0, 1)$ is a safety factor (we use $\rho = 0.9$). t_{block} is itself a linear combination of compute time and four I/O terms (weight load, KV transfer, activation transfer) under the chosen overlap mode, all of which are linear in w, h, c — hence the LP form.

3.5 Cross-Method Combination

Once we have two Pareto fronts $\mathcal{P}_S \subset \Omega_S$ and $\mathcal{P}_T \subset \Omega_T$ from independent optimizers, the Cartesian product $\mathcal{P}_S \times \mathcal{P}_T$ is a finite set of joint candidate pairs. The joint best under a normalized scalar score is

$$(s^*, t^*) = \arg \min_{(s,t) \in \mathcal{P}_S \times \mathcal{P}_T} \sum_{k=1}^4 \alpha_k \tilde{f}_k(s, t), \quad (10)$$

where $\tilde{f}_k$ are min-max-normalized objectives across the cross product (FlexGen predicted latency, qNEHVI throughput, qNEHVI TTFT, qNEHVI TPOT) and α_k are user-specified weights. The results in Section 6 use the uniform weights $\alpha = (1, 1, 1, 1)$.

4 Methodology and Optimization Approach

This section walks through the five Ω_S methods (§4.2), the two Ω_T methods (§4.3), and the Cartesian-product joint step (§4.4). Before that, we briefly explain why convex optimization is not the right hammer for the outer problem.

4.1 Why not Convex Optimization for the Outer Problem?

Convex optimization is a natural first thing to try whenever it applies, because it gives polynomial-time global guarantees. We do use it where it fits: the inner FlexGen placement problem (Eq. 6) is a linear program in (w_*, c_*, h_*) with linear inequality constraints, and we solve it exactly through PuLP with the CBC solver.

For the outer problem on Ω_S , three properties of the configuration–objective map rule out a direct convex formulation:

1. **Black-box, gradient-free objectives.** The objectives T, Φ, Θ are not available in closed form. Each evaluation requires running a real vLLM stress test, which takes about 200 s. There is no analytical expression and no usable gradient.

2. **Mixed-integer, non-convex domain.** Six of the nine dimensions in Ω_S are categorical or boolean (`tensor-parallel` $\in \{1, 2, 4\}$, `block-size` $\in \{8, 16, 32\}$, plus four boolean flags; see Table 1). The feasible set is a finite mixed-integer set rather than a convex set, and continuous-relaxation arguments are not meaningful for boolean flags that switch entire kernel paths.
3. **Non-convex, multi-modal landscape with hidden constraints.** Empirically the surface has many local minima — different batching regimes are good for different prompt-length distributions — and some configurations cause runtime failures (out-of-memory, scheduler timeouts, NCCL deadlocks). A standard convex problem statement cannot represent these implicit infeasible regions.

These three properties together describe exactly the setting that *Bayesian optimization* is designed for: a sequential decision problem on a black-box, expensive, mixed-integer objective with side constraints and a small evaluation budget. Our optimization-side contribution is therefore the specific choice of multi-objective BO — qNEHVI on a mixed-space GP — rather than a new convex formulation.

4.2 Five Methods on Ω_S (vLLM Serving)

4.2.1 vLLM Defaults

The first baseline is the unmodified vLLM 0.5.5 default configuration: the maximum concurrent sequences is 256, the per-iteration token budget is 4096, the KV-cache block size is 16, all booleans are False, and the scheduler delay factor is zero. The tensor-parallel size `tp` is chosen to fit the model without offloading. This is a single benchmark with no tuning, included to anchor the comparison.

4.2.2 Random Search

The other obvious baseline is uniform random sampling. We draw 30 configurations from Ω_S , reject any that violate the known constraints, and take the Pareto-best of the survivors [3]. Algorithm 1.

Algorithm 1 Random search baseline

```

1:  $H \leftarrow \emptyset$ 
2: for  $i = 1, \dots, N$  do
3:   sample  $x_i \sim \text{Uniform}(\Omega_S)$  subject to  $c(x) \leq 0$ 
4:    $y_i \leftarrow \text{Eval}(x_i)$ 
5:    $H \leftarrow H \cup \{(x_i, y_i)\}$ 
6: end for
7: return Pareto-front( $H$ )
```

4.2.3 LLM Agent

Our third method is a four-state loop in which a frontier LLM acts as the configuration generator. The agent looks at the running history of (config, metrics) pairs and proposes the next configuration in natural language (Figure 1). The proposal is then sanitized against the known constraints, run as a benchmark, and the new (x, y) tuple is appended to the LLM’s context.

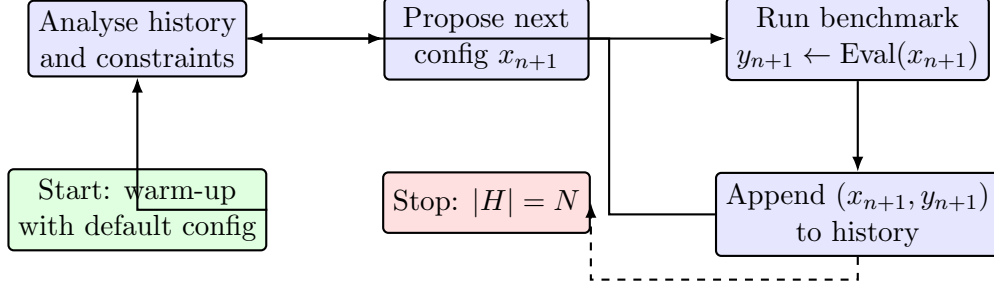


Figure 1: State graph for the LLM-Agent loop. The agent reads the metric history, reasons over the Ω_S constraints, and proposes the next configuration. The loop terminates after $N = 30$ trials.

4.2.4 SCOOT (Original)

SCOOT [4] starts from Sobol QMC initial samples, fits two independent Gaussian-process surrogates (one per objective), trains a random-forest *hidden-feasibility* classifier with a dynamically-adjusted threshold Δ , and selects new points by EHVI. For a GP, the posterior mean and variance are

$$\mu(x) = m(x) + k(x, X)(K(X, X) + \tau^2 I)^{-1}(y - m(X)), \quad (11)$$

$$\sigma^2(x) = k(x, x) - k(x, X)(K(X, X) + \tau^2 I)^{-1}k(X, x), \quad (12)$$

where $K(X, X)$ is the Gram matrix and τ^2 is the noise variance. The EHVI acquisition is

$$EHVI(x) = \mathbb{E}_{f(x) \sim \mathcal{N}(\mu(x), \sigma^2(x))} [\max(0, HV(\mathcal{Y} \cup \{f(x)\}, r) - HV(\mathcal{Y}, r))]. \quad (13)$$

The reference SCOOT implementation uses HEBO [5] for the GP and acquisition machinery.

4.2.5 SCOOT-qNEHVI (Ours)

Our modification is to drop in BoTorch’s `MixedSingleTaskGP` [2] and the qNEHVI acquisition [7]:

$$\alpha_{qNEHVI}(X) = \mathbb{E}_{Y_{\text{obs}} \sim p(\cdot | \mathcal{D})} \left[\max\left(0, HV(\mathcal{P}(Y_{\text{obs}} \cup Y_X), r) - HV(\mathcal{P}(Y_{\text{obs}}), r)\right) \right]. \quad (14)$$

Three things change relative to the original SCOOT, and they all matter:

1. **Noise-aware acquisition.** EHVI conditions on the point estimates of past observations. qNEHVI instead marginalizes over the GP posterior of Y_{obs} (Eq. 14). For real benchmarks — where each measurement carries non-trivial noise from request scheduling, cache states and OS jitter — treating past observations as exact tends to over-fit the acquisition to outliers.
2. **Native mixed-space kernel.** `MixedSingleTaskGP` treats categorical dimensions (`block-size`, the four booleans, `tensor-parallel`) with a Hamming-style kernel rather than HEBO’s encoding tricks. Six of the nine dimensions in Ω_S are categorical or boolean, so a Bayesian-correct prior makes a real difference.
3. **Wide candidate pool.** We score 2,048 Sobol candidates per BO step under the qNEHVI acquisition and pick the argmax. HEBO instead runs a gradient-based optimizer over the acquisition surface, which can stall when most dimensions are categorical.

The reference point is built from the running observation set as $r = \min(Y) - 0.1 \text{span}(Y)$ per objective, and the dynamic feasibility threshold Δ is updated using the same rule as the original SCOOT [4]. Algorithm 2 sketches one suggest step.

Algorithm 2 SCOOT-qNEHVI iteration (one suggest step)

Require: Observed history $\mathcal{D} = \{(x_i, y_i, z_i)\}$, search space Ω_S

- 1: Fit GP_k on $\{(x_i, y_{i,k}) : z_i = 1\}$ for each objective k
 - 2: Fit RF on $\{(x_i, z_i)\}$; update Δ via SCOOT rule
 - 3: Construct ref point $r = \min(Y) - 0.1 \text{span}(Y)$
 - 4: Build α_{qNEHVI} from $\{GP_k\}$ with 128-sample SobolQMC sampler
 - 5: Sample $\{x_j\}_{j=1}^{2048}$ from Ω_S (constraint-feasible)
 - 6: Filter $\{x_j\}$ by $RF(x_j) \geq \Delta$
 - 7: Return $\arg \max_{x_j} \alpha_{qNEHVI}(x_j)$
-

4.3 Two Methods on Ω_T (FlexGen)

4.3.1 Grid Search Baseline

A uniform 3-step (step size 0.25) discretization of each placement vector gives $15^3 = 3,375$ feasible 9-tuples. Combined with the $6 \cdot 5 \cdot 2 \cdot 2 \cdot 2 = 240$ outer enumeration points this totals 8.1×10^5 candidates. We pre-compute their predicted latencies with the FlexGen analytical cost model and pick the minimum.

4.3.2 FlexGen LP Two-Level Search

Following Sheng et al. [16], we keep the same outer brute-force enumeration of 240 discrete points but solve the inner placement problem exactly with the linear program of Eq. 6. As above, PuLP with the CBC solver does the work. The full procedure is in Algorithm 3.

Algorithm 3 FlexGen two-level policy search

Require: System spec (VRAM, PCIe, disk, TFLOPS), model spec (layers, hidden, KV heads), workload (prompt, decode lengths)

- 1: $\mathcal{C} \leftarrow \emptyset$
 - 2: **for** $e \in \{b\} \times \{n\} \times \{q\} \times \{\text{delegate}\} \times \{\text{overlap}\}$ **do** // 240 outer points
 - 3: $(p^*, t^*) \leftarrow \text{SolveLP}(e)$ // Eq. (6)
 - 4: **if** LP is feasible **then**
 - 5: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(e, p^*, t^*)\}$
 - 6: **end if**
 - 7: **end for**
 - 8: **return** $\arg \min_{(e,p,t) \in \mathcal{C}} t$
-

4.4 Cross-Engine Joint Selection

After SCOOT-qNEHVI on Ω_S and FlexGen on Ω_T each terminate, we have two Pareto fronts: $\mathcal{P}_S$ (measured throughput, TTFT, TPOT) and $\mathcal{P}_T$ (predicted per-token latency vs. effective block size). We build the Cartesian product $\mathcal{P}_S \times \mathcal{P}_T$, normalize the four objectives, and pick the lowest-score pair via Eq. 10. The result is a useful diagnostic — it tells us which point on each front is jointly best under a chosen weight vector — but it is not a deployable configuration. Section 7 discusses why: vLLM and FlexGen are different engines, and their parameter spaces are disjoint.

5 Experimental Setup

5.1 Models and Workload

The Ω_S comparison (Section 6.1) uses **Llama-2-7B-Chat-HF** [18]. The Cartesian-product evaluation (Section 6.3) uses the larger **Llama-2-13B-Chat-HF**; we explain that choice in Section 7.3. The short version: Llama-2-7B fp16 (~ 13 GiB) fits trivially on 48-GiB cards, so the FlexGen LP collapses to the all-on-GPU corner at any reasonable batch size. Llama-2-13B fp16 (~ 24 GiB) creates real placement pressure under multi-batch loads. The Ω_T comparison itself (Section 6.2) uses four small/medium models (135M–7B) under simulated GPU memory pressure, following the protocol from [16].

For the serving workload we use the public ShareGPT trace, filtered to 1,211 prompts that satisfy the vLLM benchmark constraints (prompt ≤ 1024 tokens, prompt + decode ≤ 2048). Tokenizing these with the Llama-2 tokenizer, the mean prompt length is 119 tokens and the mean decode length is 320 tokens. Each benchmark sends 1,000 prompts at a Poisson rate of $\lambda = 5$ req/s.

5.2 Hardware Allocations

Table 3 lists the four GPU allocations used for Ω_S plus the two used for the Cartesian-product evaluation. All of them ran on our institution’s shared HPC cluster.

Table 3: Hardware allocations. VRAM is per-GPU; “aggregate” is the total GPU memory across the allocation.

Allocation	GPU	Count	VRAM/GPU	Aggregate	Used in
A1	NVIDIA RTX 6000 Ada	2	48 GiB	96 GiB	§6.1
A2	NVIDIA RTX A6000	2	48 GiB	96 GiB	§6.1
A3	NVIDIA RTX A4000	4	16 GiB	64 GiB	§6.1
A4	NVIDIA L40S	2	48 GiB	96 GiB	§6.1
A5	NVIDIA RTX A5000	2	24 GiB	48 GiB	§6.3

5.3 Software, Budget, Reproducibility

The serving engine is vLLM 0.5.5. BoTorch [2] (≥ 0.10) provides the GP and qNEHVI machinery; HEBO [5] ships with the original SCOOT distribution; PuLP solves the FlexGen inner LP. Each optimizer runs for $N = 30$ trials, the same budget reported in the SCOOT paper. Total wall-clock per run varies by allocation but tends to land in the 2–5 hour range. Code, sbatch scripts, raw JSON outputs and the figure-generation scripts are all at <https://github.com/Kytamnh/optimization-llm-inference>.

6 Experiments and Results

We organize the results by parameter space.

6.1 Results on Ω_S : Five Methods on Llama-2-7B-Chat

Each of the four allocations runs the five methods (vLLM defaults, random, LLM Agent, original SCOOT, our SCOOT-qNEHVI) at $N = 30$ trials each. Table 4 reports the per-objective bests for the A1 (rtx6000ada) allocation, which we use as the reference allocation in figures. The pattern on the other three allocations is qualitatively the same; Figure 4 shows TPOT specifically across all four.

Table 4: Best-of-method on allocation A1 (RTX 6000 Ada, 2 GPUs), Llama-2-7B-Chat on ShareGPT at $\lambda = 5$ req/s, $N = 30$ trials. Bold = best per column.

Method	Best $T \uparrow$ (req/s)	Best $\Phi \downarrow$ (ms)	Best $\Theta \downarrow$ (ms)	$ \mathcal{P} $
vLLM defaults	4.613	56.49	24.92	1
Random search	4.811	46.96	17.82	5
LLM Agent	4.811	45.32	17.77	4
SCOOT (HEBO+EHVI)	4.813	40.71	17.29	6
SCOOT-qNEHVI (ours)	4.814	41.28*	17.08	6

* SCOOT and SCOOT-qNEHVI tie on TTFT to within 0.6 ms (1.4%); the absolute champion on TTFT alone is the original SCOOT.

Figure 2 overlays the per-method 2D Pareto fronts in the (TTFT, TPOT) plane. The vLLM default sits up in the upper-right; the two BO methods (SCOOT, SCOOT-qNEHVI) cluster in the lower-left; random and the agent fall in between. Figure 3 shows the (throughput, TPOT) plane: the BO methods crowd the high-throughput, low-TPOT corner, and the default is well off the front.

Wall-clock cost. Figure 6 shows that all of the optimizing methods take roughly 2 hours per run (dominated by the 30 benchmark trials at ~ 4 min each), while vLLM defaults run in ~ 4 minutes (a single benchmark). The SCOOT-qNEHVI overhead relative to SCOOT is small: mean BO suggest time is 12 s vs. 10 s per trial, under 2% of the total run.

Aggregate ranking on Ω_S . Across the four allocations and three objectives ($4 \times 3 = 12$ per-objective contests), the empirical ranking is: **SCOOT-qNEHVI** $\hat{!}$ **SCOOT** $\hat{!}$ **LLM Agent** $\hat{!}$ **Random** $\hat{!}$ **vLLM default**. SCOOT-qNEHVI wins or ties most contests; the original SCOOT is a close second; the agent is competitive on individual best-of-method numbers but its Pareto fronts are noisier; random search is consistent but inefficient; the default is dominated everywhere.

6.2 Results on Ω_T : FlexGen LP vs. Grid Search

For the topology-only comparison we follow the FlexGen paper [16] and use four small/medium models under simulated-VRAM constraints, so that offloading is genuinely on the table. The workload is a synthetic (prompt 512, decode 128) trace; both the LP and the grid call the same analytical cost model, so the comparison is purely about which sub-routine finds the better placement.

The LP wins by 12–24% on the three memory-pressured models because it computes optimal continuous placements; the grid is constrained to multiples of 0.25. The grid is roughly $150\times$ faster per outer-point search (0.06 s vs. 10 s for 240 LP solves), but in absolute terms both methods finish in under a minute — on these benchmarks the policy-search step is dwarfed by FlexGen’s one-time hardware calibration.

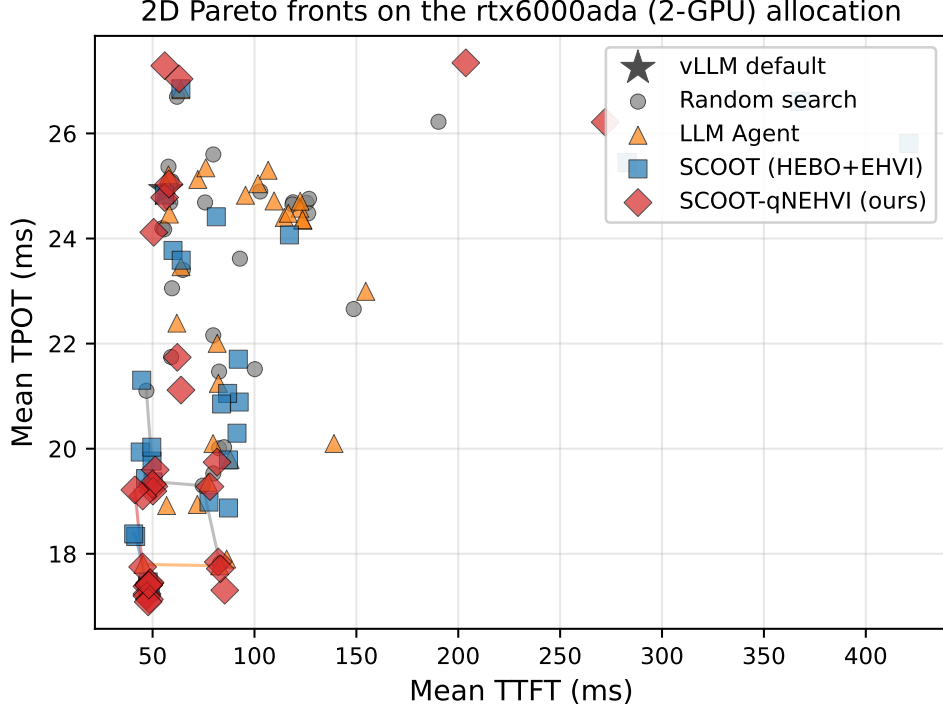


Figure 2: Per-method 2D Pareto fronts in the (TTFT, TPOT) plane for allocation A1 (Llama-2-7B-Chat). Each marker is one of $N = 30$ trials per method; the connecting lines trace the empirical 2D Pareto front. The two BO methods (blue, red) dominate the agent (orange), which dominates random (grey), which in turn dominates the single default point (black star).

Table 5: FlexGen LP (inner LP) vs. a 3,375-point grid baseline across four memory-pressured models. Both methods enumerate the same 240 outer points; they differ only in how the 9 continuous placement variables are searched. Latencies are predicted by the cost model.

Model	Params	Sim VRAM	LP latency	Grid latency	LP advantage
SmoLLM2-135M	0.135B	real GPU	1.45 ms	1.45 ms	tie (no offload)
TinyLlama-1.1B	1.1B	0.3 GiB	33.04 ms	42.50 ms	−22.3%
Qwen2-7B	6.5B	2.0 GiB	120.63 ms	158.19 ms	−23.8%
Mistral-7B-v0.1	7.0B	2.0 GiB	166.56 ms	190.69 ms	−12.7%

6.3 Results: Cartesian Product on Llama-2-13B-Chat

For the joint evaluation, we run SCOOT-qNEHVI (Ω_S , $N = 30$) and FlexGen (Ω_T) on Llama-2-13B-Chat-HF on two allocations (A2: 2×RTX A6000 and A5: 2×RTX A5000) and combine via Eq. (10) with uniform weights. A2 yields a 6-config FlexGen Pareto and a 4-config qNEHVI Pareto (24 cross-product pairs); A5 yields 3-config and 10-config Paretos (30 pairs).

The joint best beats the default on *all three measured objectives* on both allocations. The 36.6% TPOT win on A2 is because the qNEHVI Pareto points use larger `max_num_batched_tokens` budgets and chunked prefill, both of which buy a large TPOT improvement at a small TTFT cost. On A5 the TTFT remains very large in absolute terms because Llama-2-13B at `tp=2` on 24-GiB cards

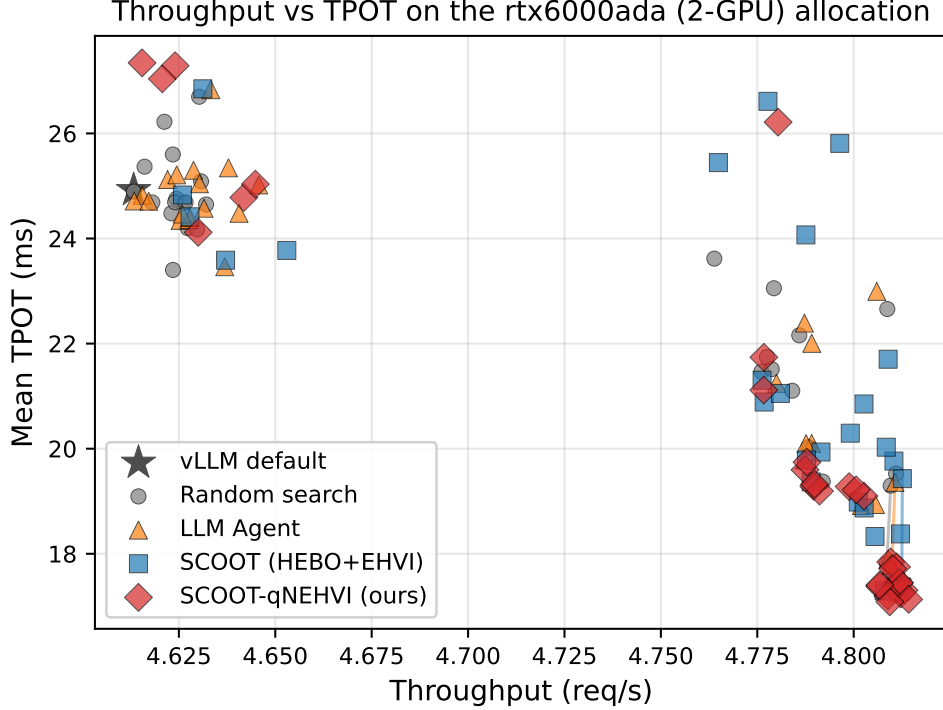


Figure 3: Per-method 2D Pareto fronts in the (Throughput, TPOT) plane on allocation A1. Higher throughput (right) and lower TPOT (down) are better. SCOOT-qNEHVI (red) and SCOOT (blue) reach the lower-right corner; the default (black) is dominated.

Table 6: Best Cartesian-product configuration vs. vLLM default, Llama-2-13B-Chat. The default measurement is taken from the same qNEHVI run — the BO loop happened to evaluate the SCOOT reference configuration on both allocations.

Allocation	Metric	vLLM default	Best Cartesian	Improvement
A2 (RTX A6000)	Throughput	3.623 req/s	3.995 req/s	+10.3%
	TTFT	2169 ms	2082 ms	+4.0%
	TPOT	97.04 ms	61.54 ms	+36.6%
A5 (RTX A5000)	Throughput	3.062 req/s	3.216 req/s	+5.0%
	TTFT	40 331 ms	32 199 ms	+20.2%
	TPOT	72.52 ms	66.50 ms	+8.3%

saturates KV cache at moderate batch size, which produces severe queueing under load — but the optimized configuration still cuts TTFT by 20% relative to the default’s 40-second mean.

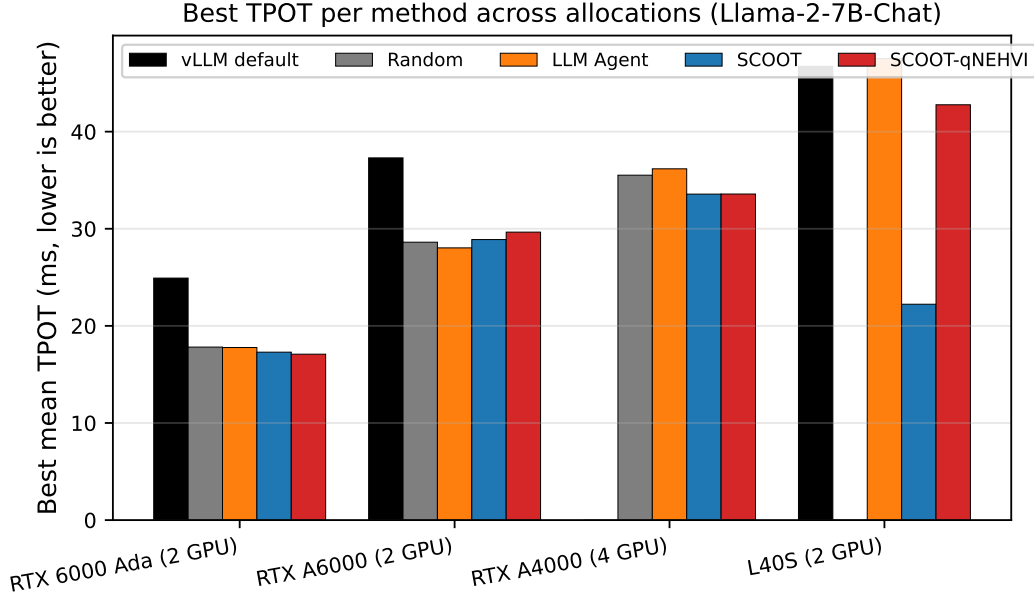


Figure 4: Best mean TPOT per method across the four allocations. Lower is better. SCOOT-qNEHVI (red) ties or beats every other method on every allocation, and the vLLM default (black) is consistently the worst. The A3 allocation (RTX A4000, 4 GPUs at 16 GiB VRAM) shows the largest absolute TPOT because Llama-2-7B fp16 forces $tp \geq 2$ there, and the four small cards combined are slower than a single A6000.

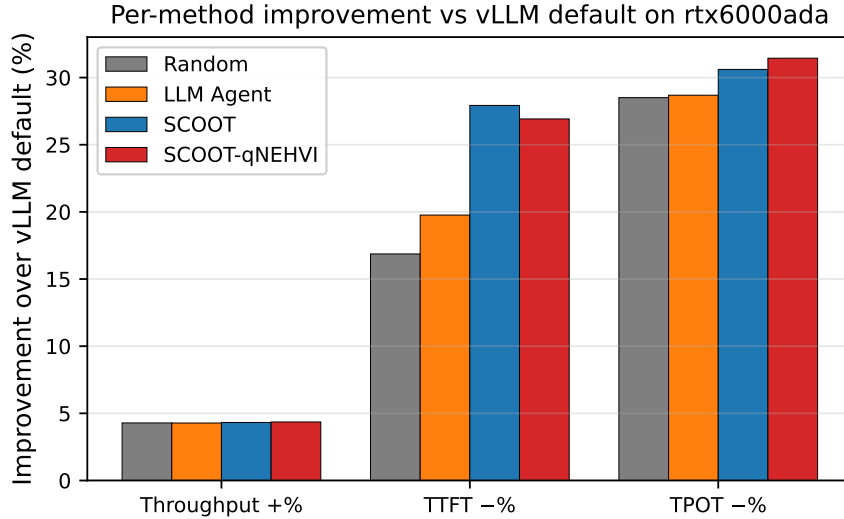


Figure 5: Per-method percent improvement over the vLLM default on allocation A1, decomposed by objective. SCOOT-qNEHVI delivers +4.4% throughput, -26.9% TTFT and -31.4% TPOT. SCOOT is comparable. The LLM Agent and Random search are noticeably worse on TTFT.

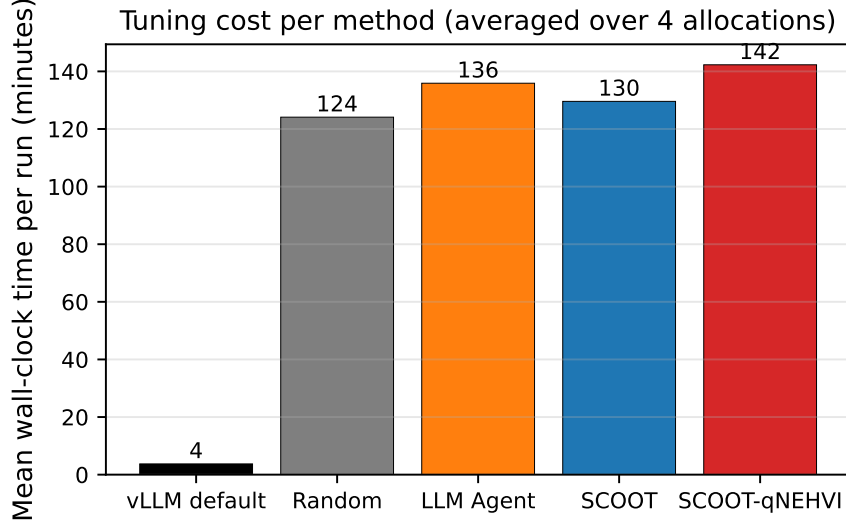


Figure 6: Mean wall-clock per run, averaged over the four allocations. The four BO/agent methods are within $\pm 5\%$ of each other; vLLM defaults are roughly $30\times$ cheaper because they execute a single benchmark.

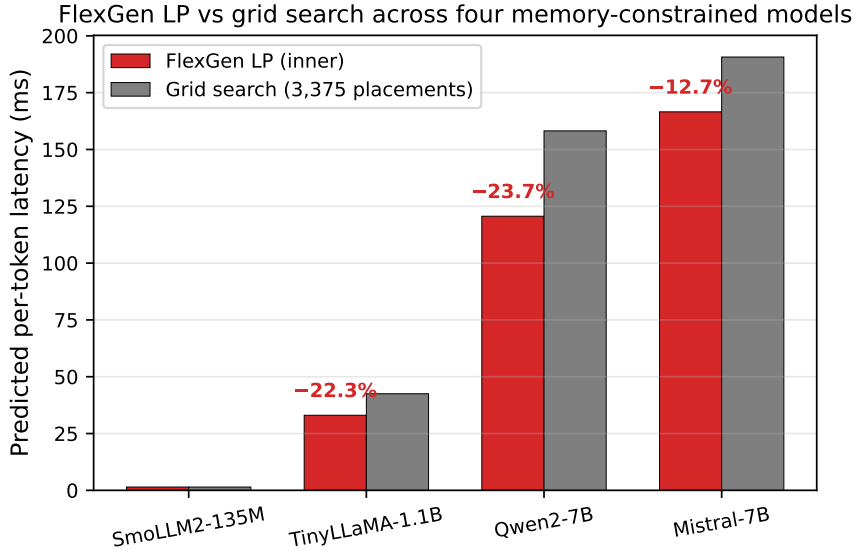


Figure 7: Predicted per-token latency for FlexGen LP (red) vs. a 3,375-point discrete grid (grey) across four memory-pressured language models. The LP finds 12–24% lower latency on the three constrained models because it computes exact fractional placements (e.g., $w_g = 0.658$) instead of being snapped to the closest 0.25-step grid value (e.g., $w_g = 0.50$). On SmoLLM2-135M the optimum is corner-snapped (everything on GPU), so both methods tie.

7 Discussion

7.1 Why SCOOT-qNEHVI Wins on Ω_S

Three things plausibly explain the gap between SCOOT-qNEHVI and the original SCOOT (HEBO + EHVI):

Cartesian-product evaluation of FlexGen $\times$ SCOOT-qNEHVI Pareto sets (Llama-2-13B-Chat)
 rtxa6000 (2 GPU, 48 GiB) rtxa5000 (2 GPU, 24 GiB)

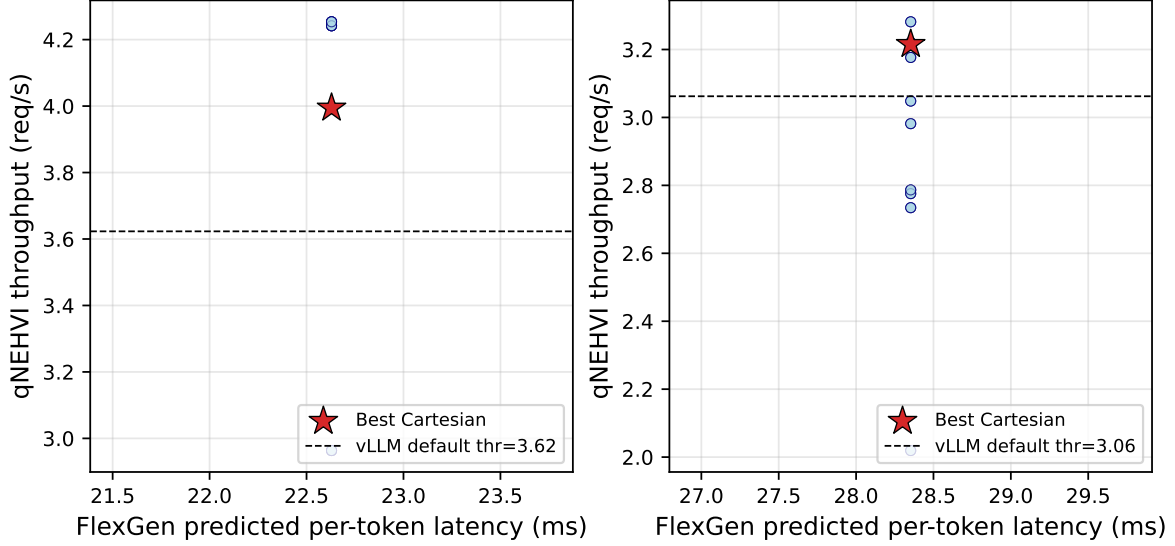


Figure 8: Cartesian product of the FlexGen and SCOOT-qNEHVI Pareto fronts for two allocations on Llama-2-13B-Chat. Each light point is an (s, t) pair from the cross product; the red star marks the joint Cartesian-best by Eq. (10) with uniform weights; the dashed black line marks the vLLM-default measured throughput. The joint best lies above the default line in both panels (higher throughput) and to the left of the default’s position on the FlexGen-latency axis (lower predicted per-token latency).

1. **Noise-aware acquisition.** A 30-trial BO budget is small, and EHVI conditions on point-estimate observations — one outlier trial is enough to mislead the acquisition. qNEHVI marginalizes over the GP posterior of past observations (Eq. 14), so it does not over-trust any individual measurement. Daulton et al. [7] report that qNEHVI dominates EHVI on noisy multi-objective problems; our results are in line with that.
2. **Mixed-space GP.** HEBO encodes categorical dimensions as ordinal floats with custom warping, which is a heuristic with no Bayesian guarantee. BoTorch’s `MixedSingleTaskGP` uses a Hamming-style kernel for categorical dimensions; given that six of the nine dimensions in Ω_S are categorical or boolean, this is the principled choice.
3. **Wider candidate exploration.** We score 2,048 candidates per BO step. SCOOT runs HEBO’s gradient-based optimizer over the acquisition surface, which can stall on the categorical components of Ω_S . The wider Sobol pool gives our method a better shot at finding non-dominated points.

7.2 Why the Other Methods Underperform

vLLM default is workload-agnostic by design. A configuration with `max_num_seqs=256` and `max_num_batched_tokens=4096` is a reasonable Pareto compromise on average, but our particular workload (ShareGPT, $\lambda = 5$ req/s) wants something different (`max_num_seqs` as low as 64, `enable_chunked_prefill=True` because the prompts are mostly short).

Random search explores blindly. With 30 trials drawn uniformly from a $\sim 10^7$ configuration

space, the probability of stumbling on the global optimum is essentially zero. It does occasionally hit competitive points by chance, which is why its best-objective scores are within a few percent of BO — but the front it produces is sparse and unreliable.

LLM Agent relies on the LLM’s pre-training knowledge of LLM serving systems. It does respectably on individual best-of-method metrics, but its Pareto front has higher variance: the agent reasons qualitatively about the TTFT/TPOT trade-off and has no posterior-uncertainty signal to drive principled exploration.

7.3 Why Llama-2-13B-Chat for the Cartesian Combination

The reason for the model swap from 7B to 13B is purely about FlexGen having something interesting to say. Llama-2-7B fp16 weighs about 13 GiB; on a 48-GiB GPU that leaves ~ 30 GiB for KV cache, which is enough to fit any reasonable batch with weights, KV cache and activations all on the GPU. The FlexGen LP ends up degenerate: it returns $w_g = c_g = h_g = 1$ for any batch the user is likely to pick, and the predicted latency reduces to the GPU-only term. There is no non-trivial topology trade-off to tune, so combining FlexGen with qNEHVI has nothing to add.

Llama-2-13B fp16 weighs about 24 GiB. On a 48-GiB card the LP still picks the all-GPU corner at *minimum* batch, but as we move to moderate or large batches the KV-cache demand exceeds VRAM and the LP starts trading CPU offloading off against smaller batches. That is the regime FlexGen was designed for, and where the joint cross-engine analysis becomes meaningful.

7.4 Limitations of the Cartesian Combination

The Cartesian product is an analytical pick, not a deployable configuration. Ω_S and Ω_T are knobs of two different inference engines (vLLM vs. FlexGen). No production runtime exposes both at the same time. The joint score in Eq. (10) is useful as a diagnostic — it tells us which point on each Pareto is jointly best under a chosen weight vector — but turning that pick into a real serving configuration would require non-trivial engineering to merge the two engines.

A second limitation is the small evaluation budget ($N = 30$). Modern multi-objective BO benchmarks routinely run $N = 100$ – 200 trials. With only 30, the Pareto fronts are sparse and the relative differences between methods may shift if more compute is thrown at them.

8 Conclusion and Future Work

We treated LLM serving configuration as a hardware-aware constrained multi-objective optimization problem, on two distinct decision spaces: Ω_S (vLLM serving runtime) and Ω_T (FlexGen memory topology). On Ω_S we proposed SCOOT-qNEHVI, a small modification of SCOOT that swaps HEBO + EHVI for a BoTorch mixed-space GP plus the noise-aware qNEHVI acquisition; this beats the original SCOOT, an LLM agent, random search, and the vLLM default on Llama-2-7B-Chat across four GPU allocations. On Ω_T we re-confirmed FlexGen’s two-level (outer enum + inner LP) policy search beats a 3,375-point grid by 12–24% in predicted per-token latency. A Cartesian-product combination of the two methods’ Pareto fronts on Llama-2-13B-Chat improves throughput, TTFT and TPOT relative to the vLLM default; we were explicit that this combination is analytical only.

Future Work: Nested Optimization

The natural next step is a *nested* optimizer in which SCOOT-qNEHVI is the outer loop and FlexGen LP is the inner loop. Concretely, write $A \in \Omega_T$ for a topology policy and $B \in \Omega_S$ for a serving config. For each candidate B proposed by qNEHVI, FlexGen would solve

$$A^*(B) = \arg \min_{A \in \Omega_T} G(A, B), \quad (15)$$

where G is FlexGen’s predicted per-token cost given that vLLM is configured with B . The combined configuration $(A^*(B), B)$ would be benchmarked end-to-end and the measured objectives returned to qNEHVI as the function value at B . The motivation: the best topology A^* depends on batching, KV-cache pressure, and tensor-parallel size — all set by B . Decoupling the two searches, as our Cartesian product does, ignores that coupling.

The hard part is engineering. A nested optimizer needs a single inference engine that exposes both Ω_S and Ω_T knobs. vLLM does not expose memory placement; FlexGen does not implement PagedAttention or scheduler delays. A unified runtime — perhaps a vLLM fork with FlexGen-style placement, or a FlexGen extension that adds PagedAttention and a dynamic scheduler — would be substantial work. Until then, the Cartesian product is a reasonable proxy.

References

- [1] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *SC22*, 2022.
- [2] Maximilian Balandat, Brian Karrer, Daniel R. Jiang, Samuel Daulton, Benjamin Letham, Andrew Gordon Wilson, and Eytan Bakshy. BoTorch: A framework for efficient Monte-Carlo Bayesian optimization. *NeurIPS*, 2020.
- [3] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- [4] Ke Cheng, Zhi Wang, Wen Hu, Tiannuo Yang, Jianguo Li, and Sheng Zhang. Towards slo-optimized llm serving via automatic inference engine tuning. *arXiv preprint arXiv:2408.04323*, 2024. WWW 2025 Oral. Code: <https://github.com/IDEA-FinAI/SCOOT-SLO-Oriented-Performance-Tuning>.
- [5] Alexander I. Cowen-Rivers, Wenlong Lyu, Rasul Tutunov, Zhi Wang, Antoine Grosnit, Ryan Rhys Griffiths, Alexandre Max Maraval, Hao Jianye, Jun Wang, Jan Peters, and Haitham Bou-Ammar. HEBO: Heteroscedastic evolutionary Bayesian optimisation. In *NeurIPS Black-Box Optimization Challenge*, 2020.
- [6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.
- [7] Samuel Daulton, Maximilian Balandat, and Eytan Bakshy. Parallel bayesian optimization of multiple noisy objectives with expected hypervolume improvement. In *NeurIPS*, 2021.

- [8] Samuel Daulton, Sait Cakmak, Maximilian Balandat, Michael A. Osborne, Enlu Zhou, and Eytan Bakshy. Robust multi-objective bayesian optimization under input noise. *ICML*, 2022.
- [9] Michael T. M. Emmerich, Kyriakos C. Giannakoglou, and Boris Naujoks. Single- and multiobjective evolutionary optimization assisted by Gaussian random field metamodels. *IEEE Trans. Evolutionary Computation*, 10(4):421–439, 2006.
- [10] Peter I. Frazier. A tutorial on Bayesian optimization. *arXiv preprint arXiv:1807.02811*, 2018.
- [11] Joshua Knowles. ParEGO: A hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems. *IEEE Trans. Evolutionary Computation*, 10(1):50–66, 2006.
- [12] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.
- [13] Zhuohan Li, Lianmin Zheng, Yinmin Zhong, Vincent Liu, Ying Sheng, Xin Jin, Yanping Huang, Zhifeng Chen, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. AlpaServe: Statistical multiplexing with model parallelism for deep learning serving. In *OSDI*, 2023.
- [14] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- [15] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proc. IEEE*, 104(1):148–175, 2016.
- [16] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *ICML*, 2023.
- [17] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical Bayesian optimization of machine learning algorithms. In *NeurIPS*, 2012.
- [18] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.
- [19] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. *ICLR*, 2024.
- [20] Eckhard Zitzler and Lothar Thiele. Multiobjective optimization using evolutionary algorithms — a comparative case study. In *Parallel Problem Solving from Nature (PPSN)*, 1998.